Listing 8.4 Catching execution errors in runnable chains

```
const chainGoogle = RunnableSequence.from([
  {
    standalone_question: standaloneQuestionChainGoogle,
    original_input: new RunnablePassthrough(),
  },
]);
const executeChain = async (input) => {
  try {
    const result = await chainGoogle.invoke(
{ question: input });
    console.log('Result:', result);
  } catch (error) {
    console.error('Chain execution failed:', error);
  }
};

executeChain("What is artificial intelligence?");
```

Creates the sequence with an array of runnables that will be executed in order

Invokes the chain sequence with an object containing the input question within a try/catch block

Executes the chain while catching any errors

This will catch the execution part of the flow, which typically is used to invoke the LLM to generate responses.

To observe the try/catch in action, you can intentionally trigger an API error: simply locate the `apiKey` variable in your `execution-error-handling.js` script and temporarily change its value to an invalid string—for example, `const apiKey = 'INVALID_KEY_FOR_TESTING'`. Then run the script again from the `ch08/chat-langchain-troubleshooting` directory using `node execution-error-handling.js`, and you will see the chain-execution-failed message logged by your catch block:

```
% node execution-error-handling.js (main !) file:///Users/theo.despoudis/
workspace/Generative-AI-Web-Apps/node_modules/@langchain/google-genai/dist/
embeddings.js:89 throw new Error("Please set an API key
for Google GenerativeAI " + ^
Error: Please set an API key for Google
GenerativeAI in the environmentb variable
GOOGLE_API_KEY or in the apiKey field of the
 GoogleGenerativeAIEmbeddings constructor
```

ENABLING VERBOSE MODE IN MODELS OR TOOLS

When initializing a model like `ChatGoogleGenerativeAI` or a tool like `DuckDuckGoSearch`, you can enable verbose logging as follows:

```
const googleModel = new ChatGoogleGenerativeAI({ apiKey, verbose: true });
```

By setting `verbose: true`, all interactions with this model or tool will log detailed information about the inputs it receives and the outputs it generates. One potential negative is that this could be overly verbose if there are multiple requests happening, so you want to configure your terminal to allow for a larger log history for easier searching.